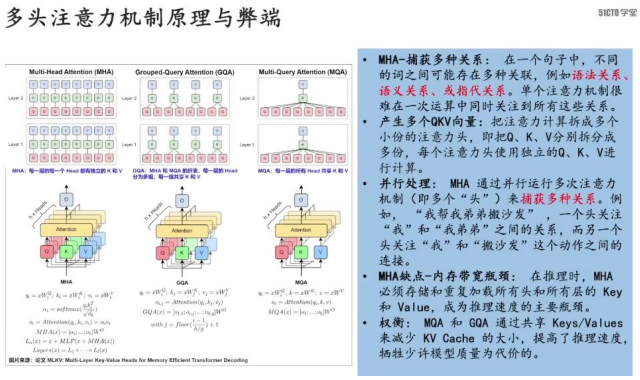
以下为AI生成的图文笔记的内容

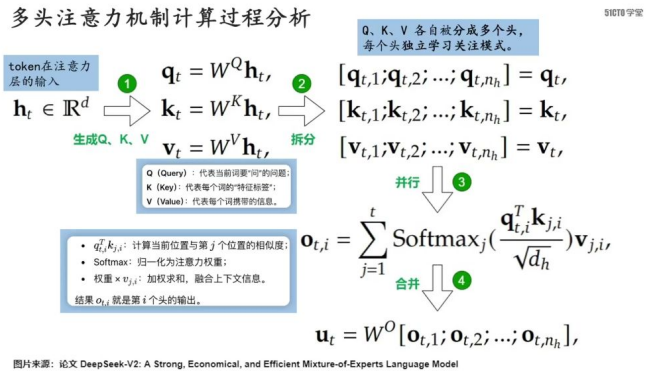
一、运行时加速方案总结 **00:05**

1. 多头注意力机制原理与弊端 00:09



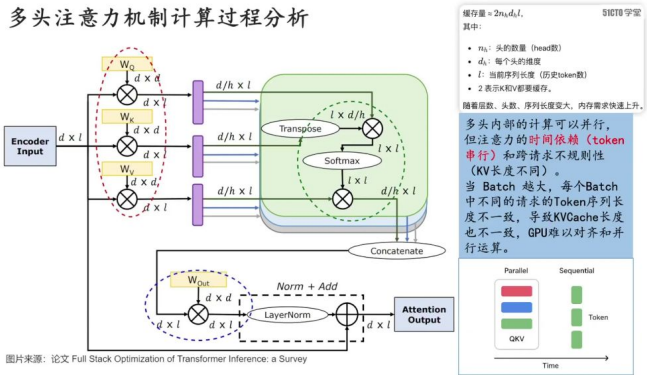
- **多种关系捕获**：在一个句子中，不同的词之间可能存在多种关联（如语法关系、语义关系或指代关系），单个注意力机制难以同时关注所有这些关系
- **并行处理机制**：通过将Q、K、V分别拆分成多份，每个注意力头使用独立的Q、K、V进行计算。例如"我帮我弟弟搬沙发"中，一个头关注"我"和"我弟弟"关系，另一个头关注"我"和"搬沙发"动作的连接
- **内存带宽瓶颈**：推理时必须存储和重复加载所有头和所有层的Key和Value，成为推理速度主要瓶颈
- **优化权衡**：MQA和GQA通过共享Keys/Values减少KV Cache大小，提高推理速度，但会牺牲少许模型质量

2. 多头注意力机制计算过程分析 00:42



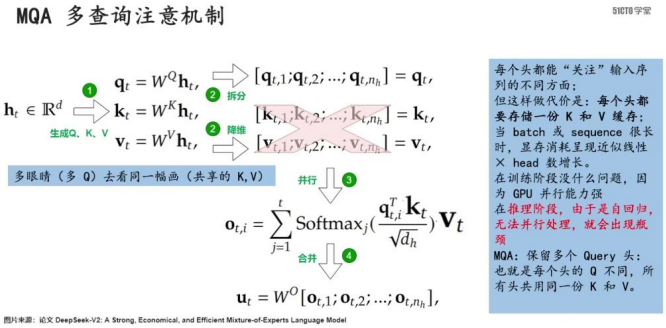
图片来源：论文 DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model

- **输入处理**：
  - 生成Q、K、V:$\mathbf{q}_t = W^Q\mathbf{h}_t, \mathbf{k}_t = W^K\mathbf{h}_t, \mathbf{v}_t = W^V\mathbf{h}_t$
  - 拆分多头:$\left[\mathbf{q}_{t,1};\ldots;\mathbf{q}_{t,n_h}\right] = \mathbf{q}_t$, 同理处理K和V
- **注意力计算**：
  - 相似度计算:$q_{t,i}^T k_{j,i}$计算当前位置与第j个位置的相似度
  - 权重归一化: 通过Softmax归一化为注意力权重
  - 加权求和:$\mathbf{o}_{t,i} = \sum_{j=1}^{t} Softmax_j\left(\frac{\mathbf{q}_{t,i}^T\mathbf{k}_{j,i}}{\sqrt{d_h}}\right)\mathbf{v}_{j,i}$
- **输出合并**:$\mathbf{u}_t = W^O\left[\mathbf{o}_{t,1};\ldots;\mathbf{o}_{t,n_h}\right]$



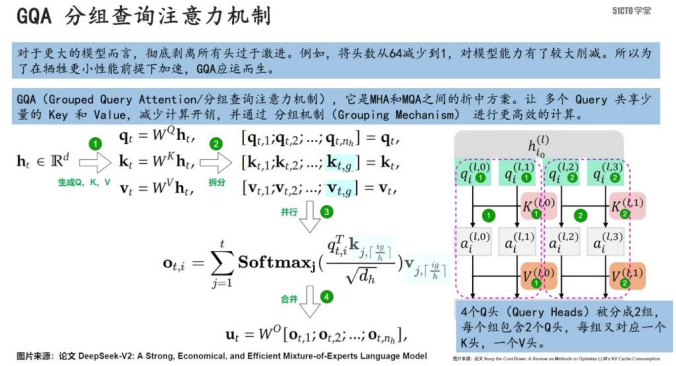
-
- **缓存量计算**: 约$2n_h d_h l$，其中n_h为头数，d_h为每个头维度，l为序列长度
- **内存需求增长**: 随层数、头数、序列长度变大而快速上升
- **并行限制**: 注意力时间依赖(token串行)和跨请求不规则性(KV长度不同)限制并行效率

3. MQA多查询注意力机制 01:17



-
- **核心思想**: 保留多个Query头(每个头的Q不同)，所有头共用同一份K和V
- **计算过程**:
  - $\mathbf{o}_{t,i} = \sum_{j=1}^{t} Softmax_j\left(\frac{\mathbf{q}_{t,i}^T \mathbf{k}_t}{\sqrt{d_h}}\right)\mathbf{v}_t$
  - $\mathbf{u}_t = W^O\left[\mathbf{o}_{t,1};\ldots;\mathbf{o}_{t,n_h}\right]$
- **优势**: 大幅减少KV Cache大小，提高推理速度
- **适用场景**: 适用于较大的模型，但对模型准确率会有一定影响

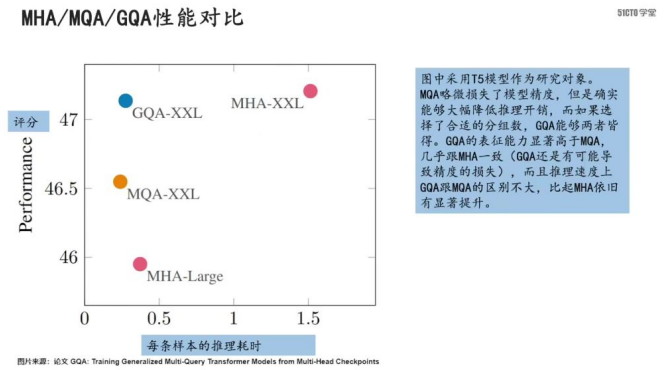
4. GQA分组查询注意力机制 01:52



-
- **设计理念**: MHA和MQA的折中方案，多个Query共享少量的Key和Value
- **分组机制**:
  - 将Q头分成多组，每组包含多个Q头
  - 每组对应一个K头和V头
  - 例如4个Q头分成2组，每组2个Q头共享1个K头和V头

- **计算优势**: 在保证效率的同时，准确率接近MHA，显著高于MQA
- **适用性**: 对模型准确率有一定要求的场景

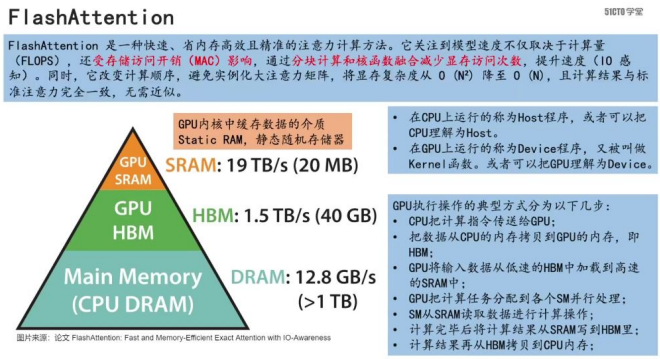
5. MQA、GQA与MHA性能对比 02:21



- **精度表现**:
  - MQA略微损失模型精度
  - GQA表征能力显著高于MQA，几乎跟MHA一致
- **推理速度**:
  - GQA跟MQA区别不大，比起MHA有显著提升
  - 选择合适分组数时，GQA能兼顾精度和速度

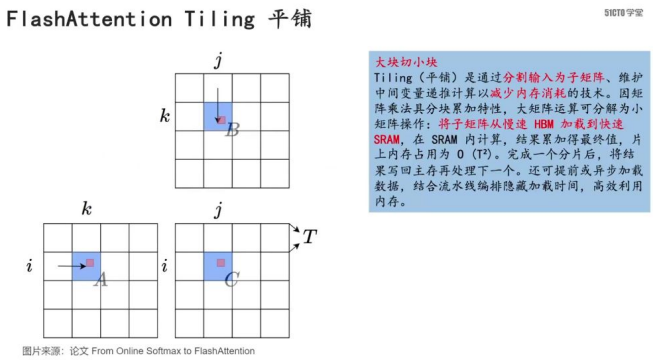
6. FlashAttention 02:32

1）FlashAttention存储分布与数据流转 02:41



- **存储层次**:
  - HBM: 高带宽内存(1.5TB/s, 40GB)
  - SRAM: 静态随机存储器(19TB/s, 20MB)
- **数据流转问题**: 传统注意力计算需要在HBM和SRAM间频繁传输数据

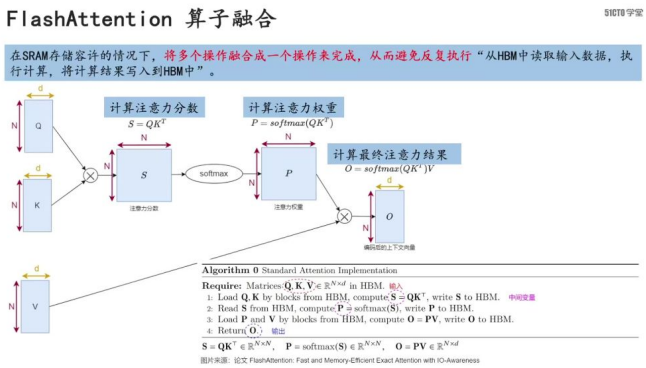
2）FlashAttention平铺 03:22



- **分块计算**: 将大矩阵运算分解为小矩阵操作
- **技术优势**:

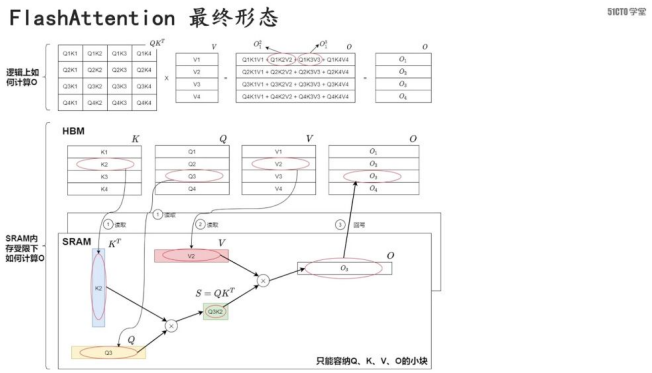
- 子矩阵从HBM加载到SRAM计算
- 结果累加得最终值
- 片上内存占用为$O(T^2)$
- 结合流水线编排隐藏加载时间

3）FlashAttention算子融合 03:48



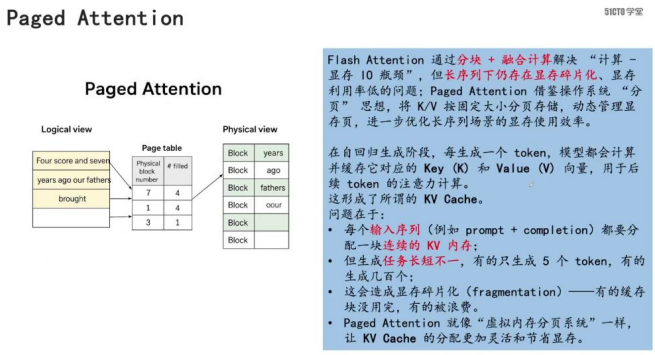
- **融合原理**: 在SRAM存储容许情况下，将多个操作融合成一个操作
- **传统流程**:
  - 计算$S = QK^T$并写回HBM
  - 读取S计算$P = softmax(S)$并写回HBM
  - 加载P和V计算$O = PV$并写回HBM
- **融合优势**: 避免反复执行"HBM读取-计算-结果写回"流程

4）FlashAttention最终形态 04:17

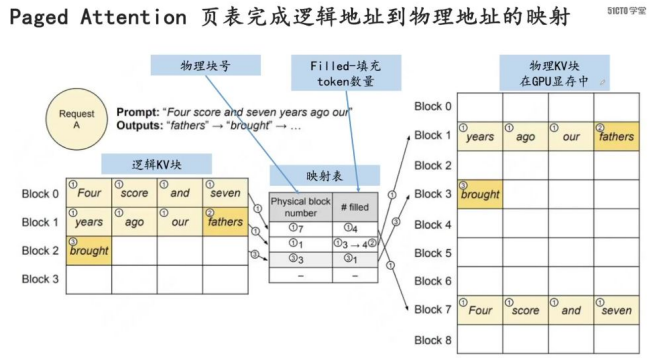


- **计算优化**:
  - 将QK^TV计算切分为小块
  - 中间结果保留在SRAM
  - 最终结果一次性写回HBM
- **效率提升**: 减少HBM与SRAM间数据传输，提高运算效率

7. Paged Attention 04:29

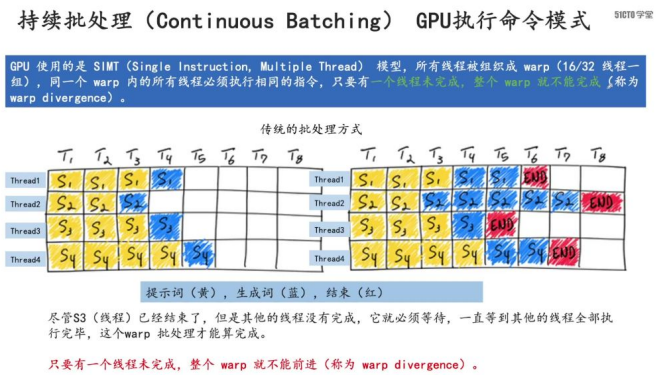


- **设计思想**: 借鉴操作系统"分页"思想，将K/V按固定大小分页存储
- **解决痛点**:
  - 传统KV Cache需要连续内存分配
  - 生成任务长短不一造成显存碎片化
- **物理实现**: 通过页表完成逻辑地址到物理地址的映射



-
- **显存复用**: 物理块是共享池，多个请求按页表分配
- **动态增长**: 请求生成更长时只需映射新物理页
- **并行优势**: 多个请求的KV Cache可同时驻留显存

8. 持续批处理 05:05



-
- **传统批处理问题**:
  - 采用SIMT模型，线程组织成warp(16/32线程一组)
  - 同一warp内所有线程必须执行相同指令
  - 只要有一个线程未完成，整个warp就不能前进(warp divergence)
- **持续批处理优势**:
  - 线程结束后可释放资源
  - 持续为其他线程提供SP服务
  - 最大化利用流处理器资源

9. 核心推理架构选型 05:57



框架	技术定位	核心优化机制	典型使用场景
vLLM	通用推理引擎	PagedAttention + Continuous Batching	多租户推理、在线服务（灵活）
TensorRT-LLM	硬件级极致优化	CUDA Graph + Overlap Scheduling	部署在 NVIDIA GPU 上的企业生产环境
llama.cpp	极简跨平台	轻量化 + 量化（int4/int8）	边缘设备、低功耗笔电、本地嵌入式应用

-

- **vLLM**:
  - 核心创新: PagedAttention + Continuous Batching
  - 优势: 对科研、创业团队、AIGC服务友好
  - 场景: 多租户推理、在线服务
- **TensorRT-LLM**:
  - 核心技术: CUDA Graph + kernel-level fusion
  - 优势: 硬件亲和性极强，能最大化榨干GPU算力
  - 场景: NVIDIA GPU上的企业生产环境
- **llama.cpp**:
  - 特点: 轻量化+量化(int4/int8)
  - 优势: 可移植性高
  - 场景: 边缘设备、低功耗笔电、本地嵌入式应用



- **吞吐量**: TensorRT-LLM > vLLM > llama.cpp
- **延迟**: llama.cpp > vLLM ≈ TensorRT-LLM
- **GPU利用率**: TensorRT-LLM > vLLM > llama.cpp
- **易用性**: vLLM > llama.cpp > TensorRT-LLM

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
多头注意力机制	加强词语间多种关联性，但增加**KV Cache**存储需求	并行计算与顺序生成的矛盾	★★★
多查询注意力（MQA）	多个Q向量共享一个KV Cache，**降低存储压力**但影响准确率	仅适用于大模型	★★
分组查询注意力（GQA）	将Q头分组，每组对应独立KV Cache，**平衡效率与准确率**	分组策略对性能的影响	★★★
Flash Attention	分块运算+核函数融合，减少SRAM与显存数据交换	**分块大小优化**与内核缓存限制	★★★★
Page Attention	逻辑-物理内存映射表解决**显存碎片化**问题	连续空间处理效率优势	★★
持续批处理	线程资源动态复用，**榨干GPU流处理器资源**	WAP调度器的单命令多线程限制	★★★

推理架构选型	**vLLM**（易用）、**TensorRT**（硬件极致）、**Llama.cpp**（边缘设备）	硬件需求与场景适配性对比	★★★